AI-Powered Expense Fraud Detection System

Final Project Report

CPSC 490-12, Spring 2026

California State University, Fullerton

Team Members: Jim Alvarez & Evan Miller

Instructor: Kanika Sood

1. Abstract

This report presents Receiptly, an AI-powered expense fraud detection system developed as a senior capstone project. The Spring 2026 MVP processes receipt submissions, extracts structured OCR data, identifies duplicate and policy-related issues, calculates a reviewer-facing risk score, and provides a React interface for upload and review. The backend is operational and supported by 53 passing unit tests across the fraud scorer, policy validator, and duplicate detector. Current testing has used the SROIE Dataset V2 from Kaggle for edge case receipt analysis, while the full 150-receipt labeled benchmark set is planned for Phase 2. The platform is designed to expand into Gemini Vision fallback, behavioral anomaly detection, analytics, authentication, and production deployment during Fall 2026.

2.0 Introduction

Expense reimbursement fraud occurs when an employee submits a false, duplicate, inflated, or non-business expense for repayment. Although each claim may appear small, repeated abuse can lead to meaningful financial loss. Traditional review depends heavily on people manually checking receipts, comparing submissions against policy, and deciding which cases need follow-up.

Receiptly reduces that workload by acting as a decision-support tool. It extracts key receipt fields, runs targeted fraud checks, assigns a risk level, and presents the evidence needed for a reviewer to approve, reject, or investigate a submission. The system still keeps human judgment in the loop, but it prioritizes the receipts most likely to require attention.

The Spring 2026 milestone delivers the core MVP: backend API, database schema, OCR service, duplicate detection, policy validation, fraud scoring, review actions, Gemini-powered explanations, and a frontend prototype. The Fall 2026 phase will focus on production readiness, including advanced machine learning, analytics, authentication, deployment, and a final labeled evaluation dataset.

2.1 Problem Statement

Traditional expense review is difficult to scale because reviewers must manually interpret receipt images, compare them with past submissions, and apply company policy consistently. This project addresses that problem with an automated pipeline that highlights suspicious submissions before reimbursement. The main challenge is that fraud does not appear in only one form: duplicate images require visual comparison, excessive charges require policy validation, and unusual purchase categories may benefit from AI-based interpretation.

- Build an end-to-end receipt upload and fraud analysis pipeline.
- Extract merchant, date, amount, transaction time, and item data from receipt images using OCR.
- Detect duplicate receipts using perceptual image hashing.
- Validate extracted receipt data against configurable company policy rules.
- Combine fraud signals into a 0-100 risk score with low, medium, and high risk levels.
- Provide a frontend interface for uploading, browsing, reviewing, approving, rejecting, and re-analyzing receipts.
- Generate plain-English explanations for flagged receipts using Gemini 2.5 Flash while caching results to control cost.
- Design the architecture so advanced ML, analytics, and production features can be added in Phase 2.

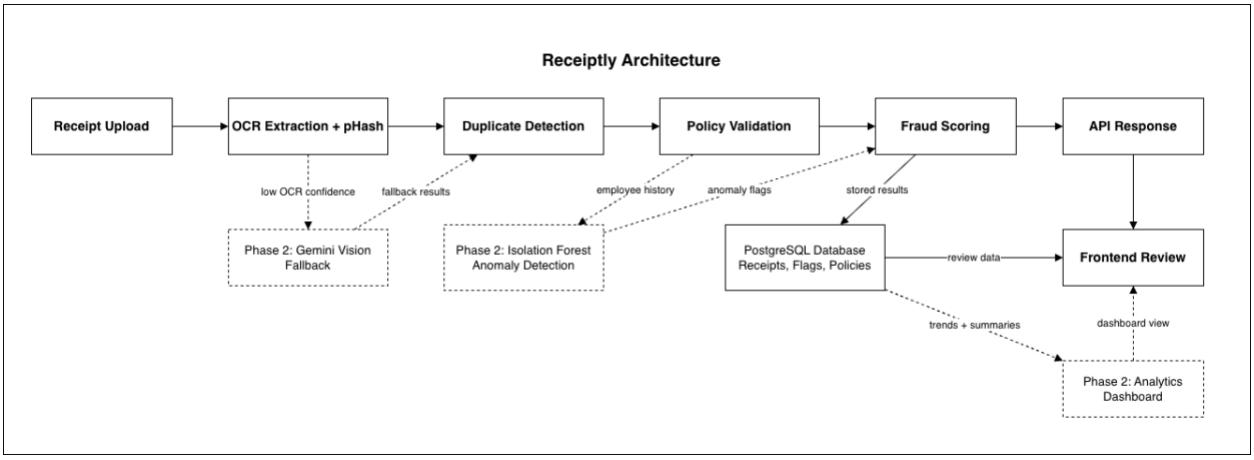
3.0 Background and Related Technology

The project uses several complementary technologies. OCR converts a receipt image into usable fields, perceptual hashing measures visual similarity between receipts, policy validation checks extracted values against company rules, risk scoring converts detector output into a single priority signal, and AI explanation summarizes the result in plain English for the reviewer.

Using separate detection layers keeps the system explainable and easier to test. Each detector can be evaluated independently, and the final result can show which specific signals contributed to a receipt being marked as low, medium, or high risk.

4.0 System Overview

The system begins when a reviewer uploads a receipt through the frontend. The FastAPI backend validates the file, performs OCR and pHash computation, stores the receipt in PostgreSQL, runs fraud checks, and returns the analyzed result to the interface. From there, the reviewer can inspect extracted fields, fraud flags, line items, risk score, and any generated explanation before making a decision.



[Figure 1 - System Architecture Diagram: Flowchart showing Upload -> OCR + pHash -> Duplicate Detection -> Policy Validation -> Fraud Scoring -> API Response -> Frontend Review. Includes Phase 2 additions for Gemini Vision fallback, Isolation Forest anomaly detection, and analytics dashboard.]

4.1 Technology Stack

Layer	Technology	Purpose
Backend API	FastAPI, Python 3.11+	Asynchronous REST API and automatic OpenAPI documentation
Database	PostgreSQL, SQLAlchemy async ORM, Alembic	Receipt storage, fraud flags, policy rules, migrations
OCR	Tesseract OCR	Local text extraction from receipt images
Image Similarity	ImageHash / pHash	Duplicate receipt detection using visual fingerprints

AI Explanation	Gemini 2.5 Flash	Plain-English explanations for flagged receipts
Frontend	React 18, TypeScript, Vite, Tailwind CSS, shadcn/ui, React Router	Reviewer dashboard, upload workflow, receipt list, receipt detail
Testing	pytest	Unit tests for scoring, policy rules, and hash computation
Phase 2 ML	scikit-learn Isolation Forest and K-means	Anomaly detection and risk clustering

Table 1. Project technology stack and purpose.

4.2 High-Level Data Flow

- The reviewer uploads a receipt image through the React frontend.
- The FastAPI upload endpoint validates the file type and size.
- The backend runs OCR extraction and pHash computation in parallel.
- Extracted fields and image hash are stored in PostgreSQL.
- The fraud detection pipeline runs duplicate detection and policy validation.
- The fraud scorer combines flags into a numerical risk score and risk level.
- The frontend displays the receipt, flags, OCR fields, and review actions.
- If the receipt has fraud flags, the reviewer can request a Gemini-generated explanation.

5.0 Backend Design and Implementation

The backend uses a services-layer design so API routes stay focused on request handling while business logic remains isolated. OCR, duplicate detection, policy validation, fraud scoring, and pipeline orchestration each live in their own modules. This reduces duplication in the codebase and makes future detectors easier to add.

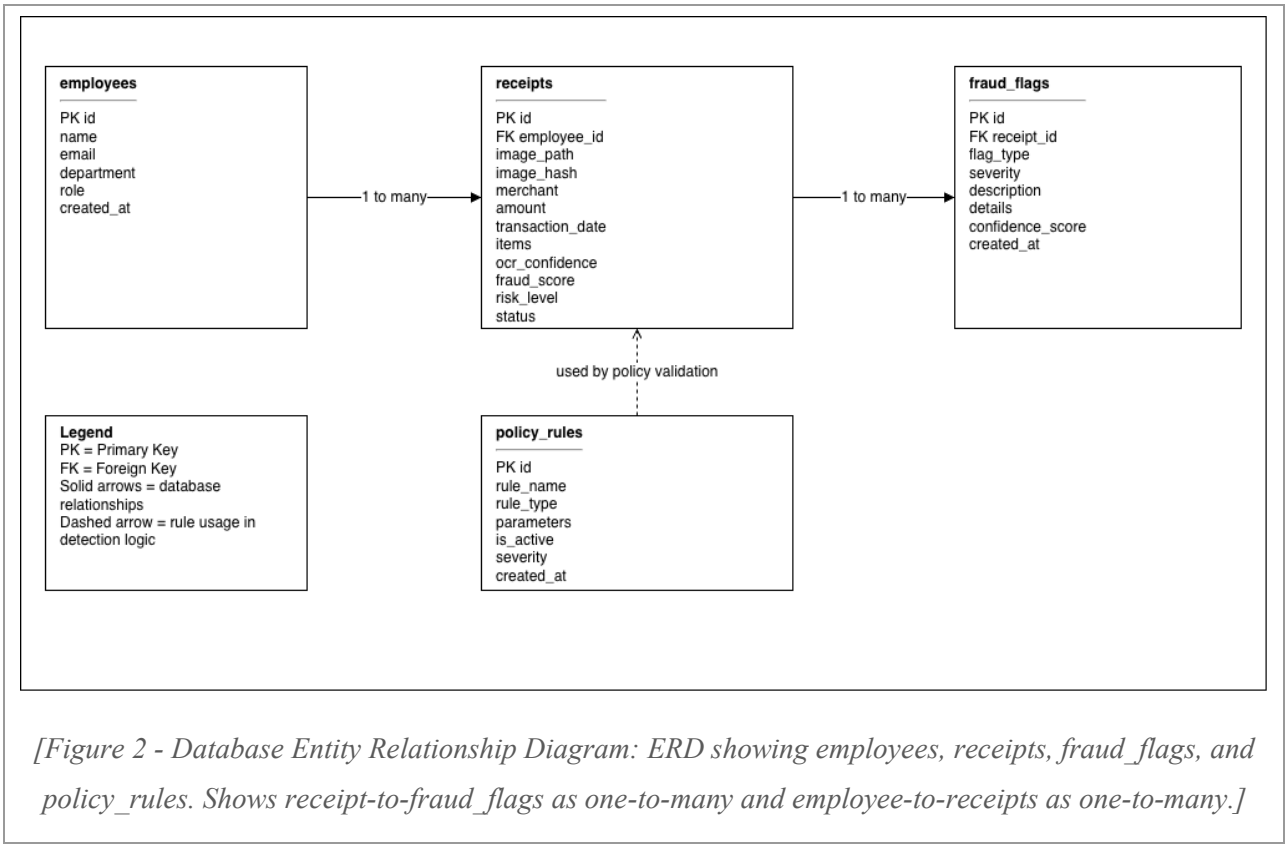
5.1 Backend Directory Structure

The project uses a domain-focused backend structure. Models define database tables, schemas define Pydantic request and response shapes, API route files expose endpoints, services contain business logic, and migrations track schema changes. The current MVP includes implemented

models for receipts, employees, fraud flags, and policy rules. It also includes service modules for OCR, duplicate detection, policy validation, fraud scoring, and full pipeline orchestration.

5.2 Database Schema

The database stores four main categories of information: employees, receipts, fraud flags, and policy rules. Receipts hold extracted OCR fields, hash values, review status, risk results, and cached explanations. Fraud flags record the individual reasons a receipt was marked suspicious. Policy rules are stored as JSON parameters so thresholds and categories can change without rewriting code.



[Figure 2 - Database Entity Relationship Diagram: ERD showing employees, receipts, fraud_flags, and policy_rules. Shows receipt-to-fraud_flags as one-to-many and employee-to-receipts as one-to-many.]

Table	Important Fields	Purpose
employees	id, name, email, department, role	Tracks employees and supports future behavioral analysis
receipts	image_path, image_hash, merchant, amount, transaction_date, items,	Stores uploaded receipts, extracted OCR data, analysis results, and review status

	ocr_confidence, fraud_score, risk_level, status, explanation	
fraud_flags	receipt_id, flag_type, severity, description, details, confidence_score	Stores individual fraud signals generated by detectors
policy_rules	rule_name, rule_type, parameters, is_active, severity	Stores configurable company policy checks as JSON rules

Table 2. Core database tables used by the MVP.

5.3 API Endpoints

The backend exposes versioned REST endpoints under /api/v1/receipts. The endpoints support receipt upload, receipt listing, receipt detail retrieval, re-analysis, review decisions, and AI explanation generation. The API also includes a health check endpoint for server status.

Method	**Endpoint**	**Purpose**
GET	/health	Returns server status
POST	/api/v1/receipts/upload	Uploads a receipt, runs OCR and pHash, runs fraud pipeline, and stores the result
GET	/api/v1/receipts	Lists receipts with pagination and optional status filtering
GET	/api/v1/receipts/{id}	Fetches one receipt with OCR data and fraud flags
POST	/api/v1/receipts/{id}/analyze	Re-runs fraud detection after clearing old flags and cached explanation
PATCH	/api/v1/receipts/{id}/review	Approves or rejects a receipt after human review
POST	/api/v1/receipts/{id}/explain	Generates or returns cached Gemini explanation for flagged receipts

Table 3. Current backend API endpoints.

6.0 OCR Processing Pipeline

Tesseract is the primary OCR engine. Before extraction, each image is converted to grayscale, upscaled to at least 1,000 pixels wide, contrast-adjusted, sharpened, and binarized. These preprocessing steps improve results on phone-captured receipts that may include faint ink, shadows, blur, or uneven lighting.

The OCR service extracts four main fields: merchant name, transaction date and time, total amount, and line items. Merchant detection uses Tesseract structured output and ranks candidate lines using line-height information, while filtering boilerplate such as URLs, payment keywords, and footer text. Date and time extraction uses regular expressions across several common formats. Total amount extraction prioritizes keyword-anchored matches such as total and amount due, then falls back to the largest detected dollar value. Line item parsing uses line-by-line patterns and stops at footer keywords such as subtotal, tax, and total to avoid including payment card lines as purchases.

Field	Extraction Strategy
Merchant	Largest-font candidate line from Tesseract structured output, boilerplate filtered, top-zone preferred
Date	Regex matching MM/DD/YYYY, YYYY-MM-DD, and month-name formats
Time	Regex matching HH:MM with optional seconds and AM/PM
Total Amount	Keyword-anchored total search with fallback to largest dollar amount
Line Items	Per-line regex before footer keywords; stores structured item descriptions and amounts
OCR Confidence	25 points per successfully extracted field: merchant, date, total, and at least one item

Table 4. OCR field extraction strategies.

7.0 Fraud Detection Pipeline

Fraud detection is handled by modular services. Each detector returns zero or more flags, and the scorer converts those flags into a final risk score and level. Because the pipeline is service-based, Phase 2 detectors such as anomaly detection or AI-based item analysis can be added without redesigning the upload workflow.

7.1 Duplicate Detection

The duplicate detector uses perceptual hashing rather than exact file comparison. A perceptual hash creates a compact fingerprint based on the visual structure of an image. This allows the system to flag receipts that are visually the same even if they were re-photographed, cropped slightly, or captured under different brightness. The MVP compares the new receipt hash against stored hashes and flags a likely duplicate when the Hamming distance is less than or equal to 10 out of 64 bits.

7.2 Policy Validation

The policy validator evaluates active rules from the policy_rules table. The MVP supports five rule types: amount limits, future-dated receipts, vendor category validation, suspicious round-number totals, and short-window duplicate checks. Because rules are stored as JSON parameters in the database, a company can update thresholds and categories without modifying backend code. A seed script provides default rules so the system works immediately after setup.

Rule Type	Example Suspicious Condition
amount_limit	A meal receipt exceeds the configured meal spending cap
future_date	The transaction date is later than the upload date
vendor_category	The merchant does not match allowed keywords for the selected category
round_number	The total is suspiciously round and above a configured threshold
short_window_duplicate	The same employee submits a similar merchant and amount within 48 hours

Table 5. Policy validation rule types implemented in the MVP.

7.3 Risk Scoring

The fraud scorer combines generated flags into a 0-100 risk score. The scoring system considers flag type, severity, and confidence. It also uses diminishing returns so that each successive flag contributes less than the previous one. This prevents several minor warnings from automatically creating a maximum score while still allowing serious or repeated issues to produce a high-risk result. Risk levels are grouped as low from 0-29, medium from 30-69, and high from 70-100.

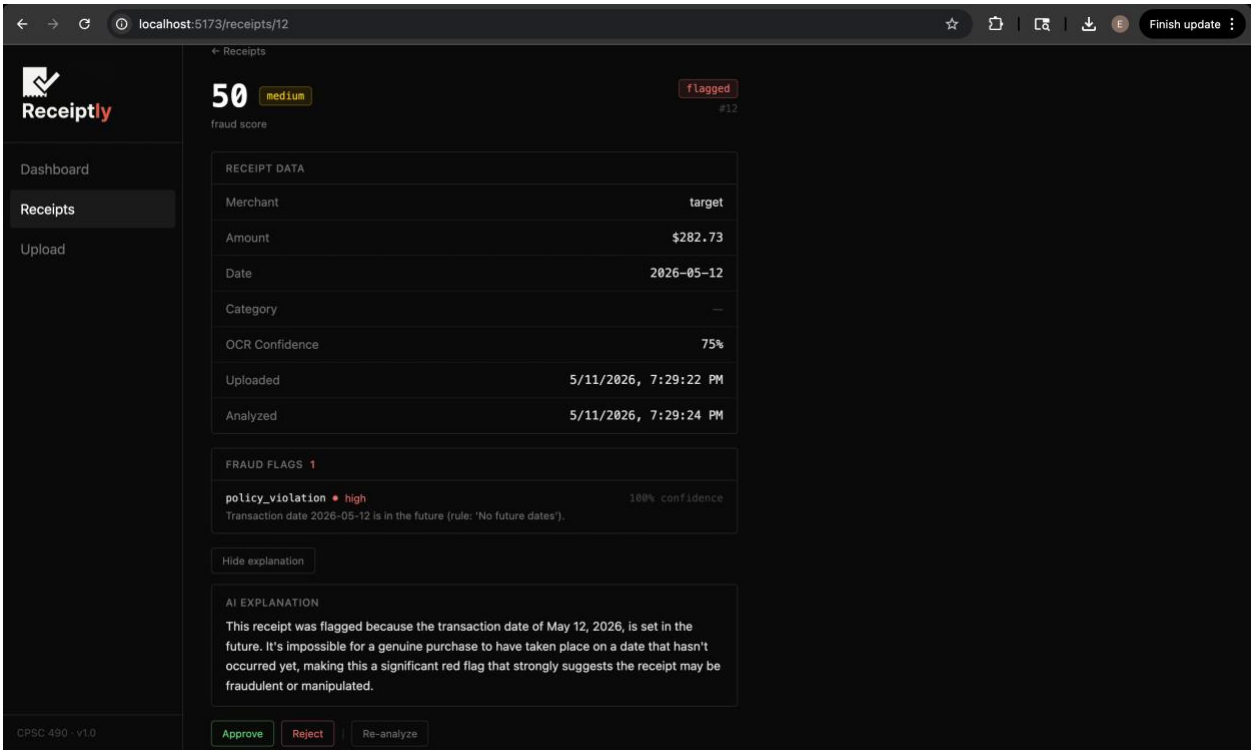
Score Range	Risk Level	Reviewer Meaning
0-29	Low	Receipt has few or no concerning signals
30-69	Medium	Receipt contains one or more issues that should be reviewed
70-100	High	Receipt has strong fraud indicators and should be investigated closely

Table 6. Risk score interpretation.

8.0 Gemini Explanation Feature

The "Why this score?" feature gives reviewers a short explanation for flagged receipts. On the receipt detail page, the button appears only when a receipt has at least one flag. When selected, the backend builds a structured prompt from the receipt fields and flags, sends it to Gemini 2.5 Flash, stores the result, and returns a concise explanation.

The feature is intentionally limited to flagged receipts because there is nothing meaningful to explain for receipts with no fraud signals. The endpoint is rate limited to five requests per day per IP during the MVP stage because authentication is not yet implemented. If the Gemini API key is missing, the endpoint returns a clear service-unavailable error instead of preventing the rest of the server from starting.



[Figure 3 – Gemini Explanation Feature Screenshot]

9.0 Frontend Design and Implementation

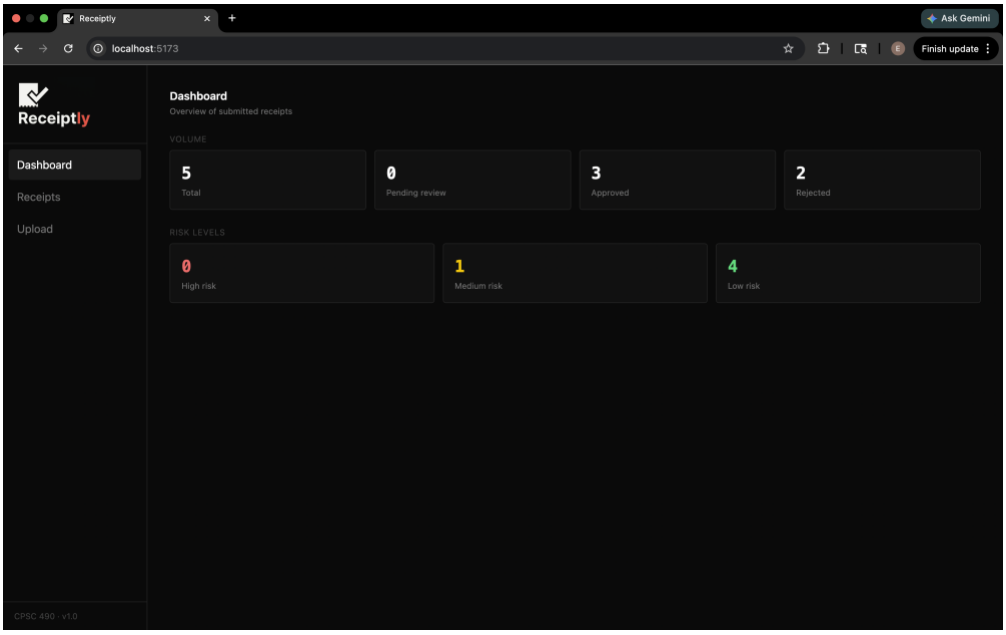
The frontend is a React 18 and TypeScript prototype built for reviewer productivity. It uses a dark fintech-style interface with sidebar navigation, compact tables, monospace formatting for IDs and amounts, and risk colors only where they help the reviewer quickly identify priority.

Page	Route	Main Functionality
Dashboard	/	Summary cards for receipt volume and risk breakdown
Receipts List	/receipts	Paginated receipt table with status filter and detail navigation
Receipt Detail	/receipts/:id	Fraud score, OCR fields, line items, fraud flags, review actions, re-analysis, explanation feature

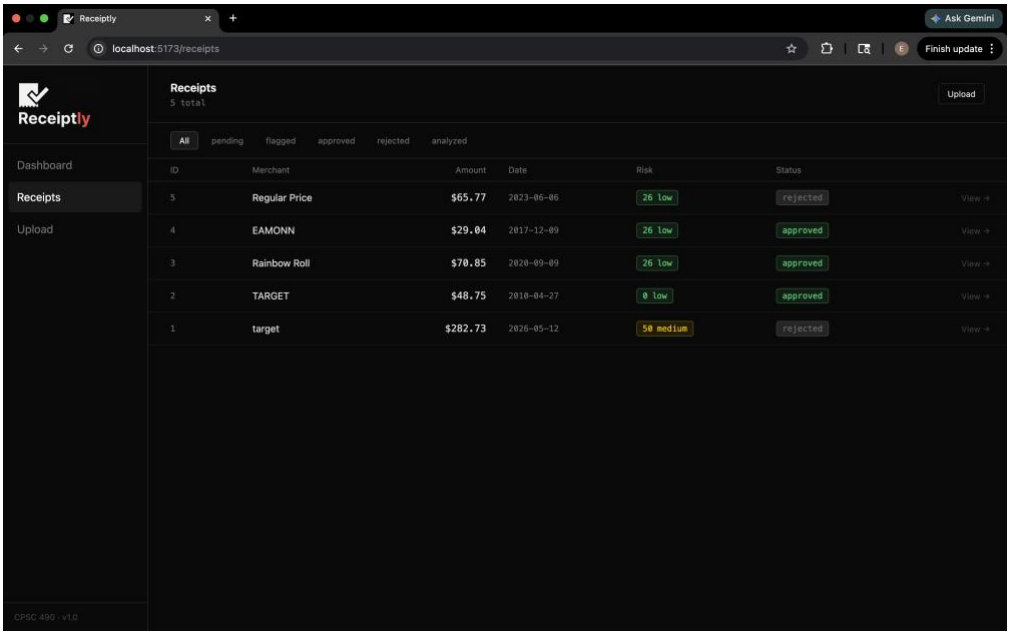
Upload	/upload	Drag-and-drop file upload with loading state and redirect to new receipt detail page

Table 7. Frontend pages included in the MVP prototype.

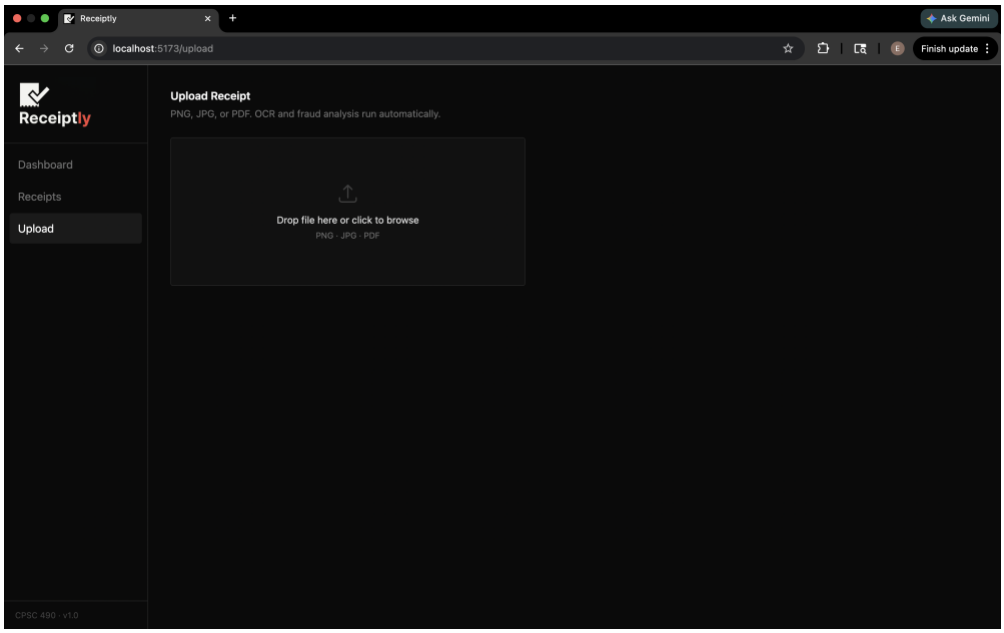
Dashboard – Showcases basic metrics for receipts within database



Receipts – Lists all stored receipts



Upload – Allows upload of pictures of receipts (runs detection pipeline automatically)



[Figure 4 - Frontend Screenshot Set: Dashboard summary, Receipts List table, Upload page]

10.0 Testing and Verification

The MVP includes 53 passing pytest unit tests. These tests verify the fraud scorer, policy validator, and duplicate detector so core fraud logic remains predictable and defensible.

Test File / Area	Number of Tests	Coverage Focus
test_fraud_scorer.py	17	Score calculation, severity weighting, diminishing returns, risk levels
test_policy_validator.py	30	Implemented policy rule types and rule edge cases
test_duplicate_detector.py	6	Hash computation and duplicate detection behavior
Total	53	Core fraud detection logic

Table 8. Current automated testing coverage.

Manual testing has verified the end-to-end upload path, including OCR execution, database storage, fraud pipeline execution, and frontend display. The team has also used the SROIE

Dataset V2 from Kaggle for edge case testing because it contains real-world receipt layouts useful for checking OCR behavior, layout variation, and difficult extraction cases. Clear receipt images generally produce usable merchant, date, amount, and item data, while blurry or low-contrast images remain a limitation that motivates the planned Gemini Vision fallback.

11.0 Results and Current Status

The Spring 2026 MVP is functional for local demonstration. Receipt upload, OCR extraction, image hashing, duplicate detection, policy validation, risk scoring, receipt listing, receipt detail retrieval, re-analysis, review decisions, Gemini explanations, and the frontend prototype are implemented through documented API endpoints.

Formal accuracy benchmarking is still pending because the full final test set has not yet been generated. The team plans to create a labeled 150-receipt dataset in Phase 2, with 100 legitimate receipts and 50 fraudulent receipts covering duplicate submissions, inflated amounts, and policy violations. For the current MVP, the SROIE Dataset V2 has been used for edge case testing rather than final scoring. Once the labeled benchmark is complete, results will be measured using precision, recall, F1-score, true positive rate, false positive rate, OCR accuracy, and total processing time.

12.0 Discussion

The strongest part of the current system is its explainable design. Instead of producing a single black-box prediction, Receiptly stores the exact flags that contributed to the risk score. This makes the output easier for reviewers to trust and easier for the team to debug.

The system also balances cost and capability. Tesseract handles the default OCR path locally, while Gemini is used only for reviewer explanations in the MVP and for planned fallback cases in Phase 2. Caching prevents repeated model calls for the same explanation.

The main limitation is that final benchmark metrics are not available yet. Because the complete 150-receipt dataset has not been generated, the team cannot make final claims about precision, recall, F1-score, or OCR accuracy. Additional limitations include the lack of authentication, production storage, role management, and real expense platform integrations, which are acceptable for the MVP but necessary before real deployment.

13.0 Project Management

The project was developed using Git for version control and a feature-branch workflow, allowing team members to work on separate parts of the system without directly affecting the main branch. Documentation was maintained across project, implementation, frontend, API, Gemini explanation, and TODO files to keep technical decisions and progress organized. The MVP work focused first on backend architecture, database models, OCR, and fraud detection services, then expanded into API documentation, frontend screens, and polish tasks. Remaining tasks include deployment, end-to-end testing with sample receipts, formal accuracy measurement, and tagging the MVP release.

14.0 Future Work

Phase 2, planned for Fall 2026, will move the project from a functional MVP toward a more complete product. Planned work includes the final 150-receipt benchmark dataset, Gemini Vision OCR fallback, Isolation Forest anomaly detection, AI-based categorization, analytics dashboards, user authentication, export tools, monitoring, and production deployment.

- Generate the final labeled 150-receipt dataset for formal precision, recall, F1-score, false positive rate, and OCR accuracy testing.
- Gemini Vision fallback when Tesseract confidence is below 70%.
- Behavioral anomaly detection using Isolation Forest.
- Employee risk clustering and fraud trend analytics using scikit-learn and dashboard charts.
- Authentication and role-based reviewer workflows.
- Production deployment on Render, Vercel, and Supabase or equivalent services.
- Export functionality for CSV or PDF review reports.
- Monitoring for false positive rate, processing time, database errors, and AI API costs.
- User profiles (Company based, each can set their own policies to look out for, etc)

15.0 Conclusion

Receiptly successfully demonstrates a functional MVP for automated receipt review. The project includes OCR extraction, duplicate detection, configurable policy validation, fraud scoring, a

documented REST API, a React frontend, and Gemini-based explanations. The system reduces manual review effort by highlighting receipts that deserve attention and presenting clear evidence for each risk score. Although final benchmarking and deployment are still pending, the implementation provides a strong foundation for Phase 2, where the team will generate the final labeled dataset and add advanced machine learning, analytics, authentication, and deployment.

16.0 References

- Association of Certified Fraud Examiners. Occupational Fraud 2024: A Report to the Nations. ACFE, 2024. https://acfe.com/report-to-the-nations/2024
- Chrome River Technologies. "Expense Fraud Costs U.S. Employers $2.8 Billion per Year." Business Wire, March 29, 2016. https://www.businesswire.com/news/home/20160329005218/en
- Ramirez, Sebastian. FastAPI Documentation. https://fastapi.tiangolo.com
- Smith, R. "An Overview of the Tesseract OCR Engine." Ninth International Conference on Document Analysis and Recognition (ICDAR 2007). IEEE, 2007.
- Google. Gemini API Documentation. https://ai.google.dev
- scikit-learn developers. scikit-learn User Guide. https://scikit-learn.org/stable/user_guide.html
- React Documentation. https://react.dev
- Kaggle. SROIE Dataset V2. https://www.kaggle.com/datasets/urbikn/sroie-datasetv2
- PostgreSQL Documentation. https://www.postgresql.org/docs/
- GitHub Repository. https://github.com/jimalvarez566/ai_fraud_payroll